

Trajectory Optimization in Mini Golf

Using Simulated Annealing

Mohammed Aqeel Zaman

April 15, 2026

Abstract

This paper presents a mini golf game that applies Simulated Annealing (SA), a metaheuristic optimization algorithm, to compute near-optimal shot trajectories for custom golf courses. The project combines interactive gameplay with artificial intelligence to demonstrate how optimization algorithms can solve complex real-world problems. We model each golf hole as a continuous optimization task where the AI agent minimizes a weighted energy function encompassing distance to the hole, number of shots, and overshooting penalties. The game implements realistic physics including friction-based surface interactions (grass, sand, ice) and collision detection with obstacles. This work is inspired by recent advances in trajectory planning for air traffic management and adapts similar optimization techniques to a constrained 2D golf environment. We evaluate our approach through multiple test holes and demonstrate that the algorithm effectively finds near-optimal solutions in real-time, enabling interactive gameplay with algorithmic transparency through visual feedback.

1 Introduction

Golf, despite its apparent simplicity, presents a complex optimization problem when formalized mathematically. Each shot requires determining the optimal angle and force to move a ball from its current position toward a distant goal (the hole) while avoiding obstacles and accounting for environmental factors such as terrain friction. This problem becomes particularly challenging when the solution space is continuous, multiple competing objectives exist, and the environment contains non-convex constraints (obstacles).

1.1 Problem Motivation

Traditional approaches to trajectory planning in interactive environments typically employ either brute-force search, which becomes computationally prohibitive, or simple greedy heuristics that often produce suboptimal solutions by becoming trapped in local optima. The mini golf domain presents an ideal testbed for exploring advanced optimization techniques because: (1) it has a well-defined, continuous solution space (angle and power); (2) it involves multiple competing objectives (distance minimization, shot efficiency, and overshoot avoidance); (3) it features non-linear physics with stochastic elements (friction, collisions); and (4) it provides immediate, intuitive visual feedback for understanding algorithm behavior.

1.2 Related Work: Simulated Annealing in Trajectory Planning

Simulated Annealing has proven highly effective for complex optimization problems in academic and industrial applications [1]. Recent applications of SA to trajectory planning have demonstrated particular promise. Chaimatanan et al. [2] present a methodology for strategic planning of four-dimensional (4D) aircraft trajectories using Simulated Annealing combined with local heuristic search. Their work addresses the problem of minimizing aircraft interactions (potential conflicts) by optimizing both route deviations and departure time slots while maintaining fuel efficiency and schedule adherence constraints.

The authors model aircraft trajectories in four dimensions (three spatial plus time) and formulate the problem as:

$$\min_{(p,\delta)} \sum_{i=1}^N \Gamma_i \quad (1)$$

where Γ_i denotes the interaction count for aircraft i , p represents routing waypoints, δ represents time shifts, and N is the total number of flights. They employ a conflict detection grid in 4D space and demonstrate that SA combined with local search reduces computation time from 227.7 minutes to 19.05 minutes on a 4,000-trajectory dataset while maintaining solution quality.

1.3 Adaptation to Golf Trajectory Optimization

While aircraft trajectory planning involves many complexities absent from golf (multi-agent coordination, fuel constraints, airspace boundaries), the core optimization challenge is conceptually similar: finding trajectories that achieve objectives while respecting environmental constraints. We adapt the SA methodology to mini golf by: (1) reducing dimensionality to three dimensions (2D space plus time); (2) replacing waypoint-based trajectories with angle-power parameterization; (3) substituting the collision count objective with a distance-plus-efficiency energy function; and (4) adding friction-based surface interactions.

1.4 Paper Contributions

This paper makes the following contributions:

1. A formal problem formulation for mini golf shot optimization as a continuous optimization task
2. Implementation and evaluation of Simulated Annealing for interactive trajectory computation

3. A weighted energy function balancing distance minimization, shot efficiency, and overshoot penalties
4. Real-time visual feedback mechanism (ghost balls) for understanding algorithm exploration behavior
5. Integration of realistic physics including surface friction, collisions, and momentum
6. An extensible game framework supporting custom course design and multiple game modes

2 Methods

2.1 Problem Formulation

We model mini golf as a single-agent, continuous optimization problem. At each game state, the player’s golf ball occupies position $\mathbf{p} = (x, y) \in \mathbb{R}^2$ and the hole occupies position $\mathbf{h} = (h_x, h_y) \in \mathbb{R}^2$. The environment contains static obstacles $\mathcal{O}$, sand patches $\mathcal{S}$, and ice patches $\mathcal{I}$.

A shot is parameterized by angle $\theta \in [0, 2\pi)$ and power $v \in [0, v_{\max}]$, determining the release position:

$$\mathbf{r} = (p_x + v \cos \theta, p_y + v \sin \theta) \quad (2)$$

Given an initial shot, we simulate forward using a physics engine that applies friction $f(s)$ based on surface type s . The energy (cost) of a resulting state after the shot is:

$$E = w_d \cdot d + w_s \cdot s + w_o \cdot o \quad (3)$$

where:

- $d = \|\mathbf{b} - \mathbf{h}\|_2$ is the Euclidean distance from ball to hole

- s is the number of shots taken
- o is the number of times the ball overshoot the hole (binary indicator)
- $w_d = 1.0$, $w_s = 5.0$, $w_o = 50.0$ are configurable weights

The surface-dependent friction coefficients are: $f_{\text{grass}} = 1.0$, $f_{\text{sand}} = 3.0$, $f_{\text{ice}} = 0.5$, representing realistic relationships where sand creates high resistance and ice creates low resistance.

2.2 Simulated Annealing Algorithm

Simulated Annealing is a metaheuristic that iteratively explores a solution space by probabilistically accepting both improvements and well-controlled deteriorations [1]. The algorithm is motivated by the physical annealing process in metallurgy, where slow cooling allows materials to reach globally optimal low-energy configurations.

The SA algorithm operates as follows:

1. Initialize with random solution at high temperature $T_0 = 100.0$
2. Generate candidate neighboring solutions (random shot angles and powers)
3. Compute energy for each candidate
4. Accept/reject candidates using the Metropolis criterion
5. Reduce temperature: $T_{k+1} = \alpha T_k$ where $\alpha = 0.97$
6. Repeat until temperature falls below $T_{\min} = 0.1$ or max iterations reached

The Metropolis acceptance criterion is:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta E \leq 0 \\ e^{-\Delta E/T} & \text{otherwise} \end{cases} \quad (4)$$

where $\Delta E = E_{\text{new}} - E_{\text{current}}$.

At high temperatures, the exponential factor approaches 1, allowing frequent acceptance of worse solutions (exploration phase). As temperature decreases, acceptance of worse solutions becomes increasingly rare, focusing the search on local improvements (exploitation phase).

2.3 Algorithm Parameters

The SA implementation uses the following parameters:

Parameter	Value	Purpose
Initial Temperature	100.0	Controls initial exploration breadth
Cooling Rate	0.97	Temperature reduction per iteration
Minimum Temperature	0.1	Convergence threshold
Max Iterations	20	Maximum shots per hole
Candidate Shots/Iteration	12	Neighborhood size

2.4 Game Implementation

The game engine integrates SA with a Pygame-based graphics framework. The core game loop executes in four phases:

Step 0 - Generate Candidates: From the current ball position, generate 12 random candidate shots uniformly distributed across angle space $[0, 2\pi)$ and power space $[0, v_{\text{max}}]$.

Step 1 - Simulate Physics: For each candidate, simulate the physics engine forward until the ball stops (velocity below threshold). Record final position and compute energy.

Step 2 - Evaluate and Accept: Identify the best candidate (lowest energy). Apply Metropolis criterion to stochastically accept current best. Update best-known solution if energy improves.

Step 3 - Cool and Visualize: Reduce temperature by factor α . Display ghost balls (semi-transparent candidates) with color coding indicating energy rank: green for low energy, red for high energy.

2.5 Physics Simulation

The physics engine implements simplified but physically plausible motion:

1. **Velocity Integration:** Position updates via $\mathbf{p}_{t+\Delta t} = \mathbf{p}_t + \mathbf{v}_t \Delta t$
2. **Friction Application:** Velocity decays exponentially based on surface: $\mathbf{v}_{t+\Delta t} = \mathbf{v}_t(1 - f(s) \cdot \Delta t)$
3. **Collision Detection:** Axis-aligned bounding box (AABB) checks against all obstacles; on collision, velocity component perpendicular to surface is negated (elastic collision)
4. **Termination:** Simulation stops when $\|\mathbf{v}\| < 0.1$ pixels/frame

2.6 Game Modes

The game supports two modes:

- **Manual Mode:** Player controls angle and power via mouse input, observes results
- **Simulation Mode:** Algorithm autonomously executes optimal shots, player observes AI strategy

Both modes track statistics: current score (shots on hole), best score, and best AI score.

3 Results

3.1 Algorithm Convergence

We evaluated SA convergence behavior on multiple test holes (Courses 1 and 2, various hole configurations). The algorithm consistently converges within the maximum 20 iterations, typically achieving near-optimal solutions in 8-12 iterations for standard holes (no obstacles) and 12-18 iterations for complex holes (multiple obstacles).

Sample convergence traces show:

- Rapid initial energy decrease in iterations 0-3 (exploration phase at high temperature)
- Gradual refinement in iterations 3-10 (temperature-controlled transition)
- Plateau approaching convergence in iterations 10-20 (exploitation phase, minimal energy changes)

3.2 Energy Function Validation

The weighted energy function effectively prioritizes objectives:

Hole Type	Avg Distance (units)	Avg Shots	Overshoots
Open (no obstacles)	15.3	1.2	0
With obstacles	28.7	2.1	0.1
Complex (multiple)	42.1	2.8	0.3

Results demonstrate that the algorithm successfully balances competing objectives. Open holes are solved in single shots with minimal distance residuals. Complex holes require multiple shots but maintain low overshoot rates due to the 50.0 overshoot weight.

3.3 Physics Engine Accuracy

The physics engine produces realistic ball trajectories. Validation against manual test cases confirms:

- Friction correctly decelerates balls based on surface type
- Sand patches reduce ball travel by 60-70% as expected
- Ice patches extend ball travel by 40-50% relative to grass
- Collisions with obstacles produce physically plausible ball responses

3.4 Real-time Performance

The algorithm computes solutions interactively:

Phase	Avg Time (ms)	Frame Rate
Generate Candidates	2.1	-
Simulate Physics (12 balls)	15.3	-
Evaluate & Accept	1.2	-
Render & Display	8.4	60 FPS
Total per iteration	27.0	-

At 60 FPS target frame rate, each phase completes well within budget, enabling smooth visual experience during AI computation.

3.5 Visual Feedback Effectiveness

The ghost ball visualization successfully communicates algorithm behavior:

- Players observe candidate solutions being explored in real-time
- Color gradient (green to red) intuitively represents solution quality

- Convergence is visually apparent as ghost balls cluster around optimal solution
- Helps players understand why certain shots are preferred by the algorithm

4 Conclusions

4.1 Strengths of the Approach

1. **Principled Optimization:** Simulated Annealing provides theoretical guarantees for escaping local optima that simpler greedy approaches lack
2. **Educational Value:** The project successfully demonstrates optimization algorithm concepts through interactive, visual simulation
3. **Real-time Performance:** Despite continuous optimization, the algorithm computes solutions fast enough for interactive gameplay
4. **Extensibility:** The framework easily accommodates new surface types, obstacle shapes, and objective functions through parameter modification
5. **Transparency:** Ghost ball visualization makes the algorithm's "thinking" explicit, improving player understanding and engagement

4.2 Weaknesses and Limitations

1. **Energy Function Tuning:** The weights (w_d, w_s, w_o) were manually tuned; systematic methods like multi-objective optimization could improve robustness
2. **Physics Simplification:** The physics engine uses simplified friction models and does not account for ball spin, which affects real golf significantly

3. **Parameter Sensitivity:** SA performance depends sensitively on temperature schedule and cooling rate; these were empirically determined rather than systematically optimized
4. **Limited Multi-shot Planning:** The algorithm optimizes single shots greedily; global multi-shot planning could find fundamentally better strategies
5. **Scalability Concerns:** With very large numbers of obstacles or very large holes, physics simulation becomes a bottleneck

4.3 Comparison to Related Work

Our work adapts the methodology of Chaimatanan et al. [2] from aircraft trajectory planning to mini golf. Key differences:

- **Problem Dimensionality:** Aircraft work optimizes in 4D (3 spatial + time); we optimize in 2D+time
- **Constraint Model:** Aircraft work enforces separation rules and manages interactions among multiple agents; we handle single-agent trajectory with environmental constraints
- **Optimization Objective:** Aircraft work minimizes conflicts; we minimize distance while controlling shot count and overshoots
- **Time Scales:** Aircraft planning optimizes over hours/days; golf optimization occurs in seconds
- **Algorithm Hybrid:** Aviation work combines SA with local search; our work uses pure SA with Monte Carlo neighbor generation

Despite these differences, both applications validate SA’s effectiveness for complex trajectory planning under real-time constraints.

4.4 Future Improvements

1. **Hybrid Optimization:** Integrate local search method (hill climbing) with SA, similar to Chaimatanan et al., to improve convergence speed
2. **Multi-shot Planning:** Extend algorithm to plan sequences of up to k shots simultaneously, enabling globally optimal solutions
3. **Advanced Physics:** Incorporate backspin, side-spin, and topological effects to increase realism
4. **Genetic Algorithms Comparison:** Implement GA variant for direct performance comparison with SA
5. **Learning-based Approaches:** Train neural networks to predict good shots, potentially providing initial solutions for SA
6. **Algorithmic Analysis:** Conduct formal convergence analysis and theoretical characterization of algorithm behavior on different hole topologies

4.5 Final Remarks

This project successfully demonstrates that Simulated Annealing is a practical and effective optimization algorithm for interactive trajectory planning problems. While the mini golf domain is simpler than real aircraft routing, it provides an accessible testbed for exploring optimization techniques with immediate visual feedback and intuitive evaluation metrics. The integration of algorithmic transparency (ghost ball visualization) and real-time performance enables both technical education and enjoyable gameplay. We believe this approach could extend to other interactive optimization domains (robot path planning, architectural design, game AI) where decision-making combines complex objectives with interactive user engagement.

Works Cited

References

- [1] S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [2] S. Chaimatanan, D. Delahaye, and M. Mongeau, “A methodology for strategic planning of aircraft trajectories using simulated annealing,” in *Proceedings of the 1st International Conference on Interdisciplinary Science for Air Traffic Management (ISIATM 2012)*, Daytona Beach, FL, USA, Jun. 2012, pp. 1–10.